

DBT_GUIDE_PLAIN

dbt Guide — In Plain English

Plain-language companion to `docs/architecture/DBT_GUIDE.md`. For the exact spec, read that / its PDF.

In one sentence

This guide shows how raw Google Analytics 4 click data gets cleaned, grouped into website visits, and turned into a few tidy, trustworthy tables that the rest of Helios reads from.

Why this matters to you

Everything downstream — the metrics, the AI agents, the final report — only ever reads these tables. If a table here is subtly wrong, every number above it is wrong too, with no error to warn you. So this is the foundation: get it right and the rest is easy; get it wrong and you debug ghosts for days. Build this layer first, and test the tricky parts hard.

The big ideas, simply

What dbt is. dbt (a tool that turns plain SQL SELECT statements into managed, dependency-ordered database tables) is the entire "transform" step. You write SELECTs; dbt builds them in the right order, tests them, and documents them.

The three layers (an assembly line). Data flows one direction only, never sideways or backward:

- `stg_*` (staging) — the *cleanup* station. One input table in, same rows out, just renamed to `snake_case`, typed correctly, light unpacking of nested fields. No joins, no math, no filtering. Built as a `view` (a saved query, no stored data).
- `int_ga4_*` (intermediate) — where the *real logic* lives. Shared rules that more than one final table needs.

- `fct_*` / `dim_*` (marts) — the *finished goods*. Wide, friendly tables the semantic layer and agents actually read. `fct_` = facts (events/measurements), `dim_` = dimensions (descriptive lookups like channels or dates).

Sessionization — the keystone. Raw GA4 is one row per *click* (event). A "session" is one *visit*. Sessionization collapses many clicks into one row per visit, grouped by the visitor + visit number. `int_ga4__sessionized` does this. If grouping is wrong, every visit-level number is quietly wrong — hence "keystone."

The grouping uses one canonical fingerprint, the `session_key`, built once by the `channel_group_case` macro's sibling, the `sessionize()` macro:

```
session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))
```

Always count visits as `COUNT(DISTINCT session_key)` — never `COUNT(*)`.

The `reached_*` flags (funnel that only narrows). The funnel is the steps a visit can pass through: `view_item` → `add_to_cart` → `begin_checkout` → ... → `purchase`. Each `reached_*` flag means "this visit hit this step *or any later one*." Because reaching a later step forces all earlier flags true, the counts can only shrink as you go deeper: `sessions` ≥ `reached_view_item` ≥ ... ≥ `reached_purchase`. That guarantees every step's conversion rate is ≤ 1 (you can't convert more than 100%). The old `did_*` flags (which meant "fired exactly this event") are retired because a deep-link purchase could skip `add_to_cart` and break the math.

Why rates are `SUM(num) / SUM(denom)`. Always add up the numerators and denominators *first*, then divide. Never average per-group rates — that's the Simpson's-paradox trap, where each group can move one way but the blended average lies. This rule is non-negotiable.

What you actually build

Build in dependency order — nothing before the things it depends on:

1. **Macros first** — the reusable SQL snippets, each a single source of truth: `get_event_param()` (pull one value out of GA4's nested params), `sessionize()` (the `session_key`), and `channel_group_case()` (buckets traffic into exactly 10 channel groups — adding an 11th is a hard error).
2. **Sources + seed** — declare the raw GA4 export as a `source` (never reference the raw table directly) plus the `channel_group_mapping.csv` seed.
3. **Staging** (`stg_ga4__events`, `stg_ga4__event_params`) — the only place that touches raw nested GA4 fields.

4. **The two keystone intermediates** — `int_ga4__sessionized` (one row per visit, with channel/source/device) and `int_ga4__funnel_steps` (the `reached_*` flags + `session_revenue`).
5. **Marts** — `core` (`fct_sessions` , `fct_funnel` , `fct_daily_funnel` , `dim_*`), `finance` (`fct_orders` , `fct_order_items`), `growth` (`fct_funnel_by_dim` , `fct_cohorts`).

Test the keystones first (golden tests). Because keystones fail silently, write the test *before* the model: feed in a handful of fake events and assert the exact expected output (two events collapse to one session; a purchase-only visit sets every `reached_*` true; a duplicate purchase counts revenue once).

Easy things to get wrong

These all fail *silently* — no error, just bad numbers:

- **Bad sessionization** — wrong key, leaking first-touch source, or dropping the landing page corrupts every downstream table.
- **Using `did_*` instead of `reached_*`** — non-monotonic flags produce conversion rates above 100%.
- **Averaging ratios** — instead of `SUM(num)/SUM(denom)` . The Simpson's-paradox bug.
- **Dropping `session_revenue`** — `fct_daily_funnel` must aggregate `fct_funnel` (which carries revenue), not `int_ga4__funnel_steps` . Lose revenue here and the eval's dollar-at-risk labels break.
- **Wrong session key** — never `FARM_FINGERPRINT` , never `COUNT(*)` . One canonical expression only.
- **`traffic_source` mix-up** — event-level `traffic_source.*` is the visitor's *first-ever* acquisition channel, not *this* visit's source. Prefer session-scoped `event_params` source/medium; fall back to `traffic_source` only when null.

Glossary — the exact words, demystified

- **dbt** — tool that builds SQL SELECTs into managed, ordered tables.
- **staging / `stg_*`** — pure rename-and-retype layer; a `view` .
- **intermediate / `int_ga4__*`** — shared business logic; usually `ephemeral` (inlined, not stored).
- **mart / `fct_*` , `dim_*`** — finished, wide tables the product reads.
- **sessionization** — collapsing clicks into one row per visit.
- **`session_key`** — the canonical visit fingerprint (the MD5 expression above).
- **`reached_*` flags** — monotonic "hit this step or later" funnel markers.

- **channel_group_case macro** — the one place traffic is bucketed into 10 channels.
- **materialization** — how a model is stored: `view`, `ephemeral`, `table`, or `incremental`.
- **incremental** — only rebuild recent day-partitions instead of all history.
- **semantic layer** — the metric registry above the marts; the agents' only data door.

When to open the real doc

Open `pdf/docs/architecture/DBT_GUIDE.pdf` (or the `.md`) when you need the *exact* SQL: the full macro bodies, the incremental `insert_overwrite` + 3-day-lookback config, partition/cluster settings, the BigQuery cost controls (`maximum_bytes_billed`, `_TABLE_SUFFIX` pruning), the full mart catalog with grains and keys, and the complete testing, freshness, and reconciliation strategy. This companion gives you the mental model; the doc gives you the copy-paste.